Verified Mechanisms

RESEARCH ENGINEER

Take-home task

- This task has two parts: make two fixed small models, Qwen and GPT-OSS, collaborate to solve math problems and prove the answers in Lean 4, then work out whether the collaboration really beats either model working alone, and why.
- Part one is scored mechanically on a private holdout set: points only for proofs the Lean compiler accepts. We clone your repository and run it ourselves.
- The deadline is **August 30** (EOD Anywhere on Earth): submit your GitHub repository link and your writeup as a **PDF** through this Google form.

Throughout this brief you will work with exactly two fixed models: **Qwen** (`qwen/qwen3.5-flash-02-23`) and **GPT-OSS** (`openai/gpt-oss-120b`). Both come through OpenRouter, with no `:free`, `:online` or other variant suffix.

The task comes in **two parts**.

- Part one is about **making Qwen and GPT-OSS collaborate**. They have to solve math problems and prove the answers in Lean 4.
- Part two is about **a scientific understanding of that collaboration**: whether it really does beat either model working alone, and where one model fills the other's gaps.

Instructions

- **What should the writeup look like?** Anywhere between 1 and 10 pages. The appendix does not count towards this limit. The writeup must explain your harness design choices, results, and the scientific understanding from part two. We grade **clarity of the writeup** as well, so budget some time for the writing itself.
- **Coding guidelines.** Your agent goes in `submission/agent.py`, and `docs/AGENT_API.md` spells out the interface it has to implement. The conditions you are judged under are in `RULES.md`. Run `scripts/judge_check.sh` before submitting; it catches most of what would otherwise cost you points.
- **OpenRouter API key.** The key arrives in the same email as this PDF with a **$50 budget, on us**. It is a hard cap, so the key stops working once it is spent rather than running up a bill. The task is designed to fit well inside it. You are welcome to buy your own credits, though we cannot reimburse that at the moment.
- **Using LLMs.** You may, and in fact we encourage you to, use LLM assistance for the analysis and for the writeup (Codex, Claude Code, whatever you normally reach for). Please disclose which models and tools you used, and how you used them in the Google form accompanying your submission.

If you have any questions, please write to `hiring@verifiedmechanisms.ai`.

Background

> Our autoresearch framework has LLM agents find theorems about the internal computations of transformers and verify them in Lean 4. We believe we can get more from models when they work together, and that how much more depends on finding the right way for them to collaborate. This task recreates the setting at a smaller scale with Qwen and GPT-OSS.

Everything you need lives in one repository:

`github.com/VerifiedMechanisms/re-takehome`

- The kit contains sixteen sample problems in `sample-problems/`, a working single-model baseline, and scripts that set up Lean and Mathlib on Docker for you.
- Build with whatever you like, including your own chatbots and coding agents; at run time your system talks only to Qwen and GPT-OSS through our key.

PART ONE

Make Qwen and GPT-OSS collaborate

Each problem is a folder: `problem.md` states it in English, `challenge.lean` states it formally, with `sorry` where the proof goes. Your system turns the folder into a Lean file the compiler accepts, with our statement unchanged.

Build the coordination layer. In this task, you are expected to design the collaboration: decide how the two models split the work and coordinate, so that, together, they solve and formally verify as many problems as possible. We would rather see a simple design we can fully understand than a complicated one that scores marginally better.

Scoring. On a private holdout set (with the same format as the sample set), we award one point per correct problem, zero for incorrect solutions. A proof counts only if it passes the open-source Lean comparator (`leanprover/comparator`). Each problem runs under a cap of **$1 and eight hours**[1] **of wall-clock time**, and an attempt that exceeds either cap scores zero even if the proof is correct.

PART TWO

Understand when collaboration helps

How does your two-model system compare with each model working alone? When collaboration helps, identify what one model contributes that the other lacks. You can start by evaluating the three conditions—**Qwen solo, GPT-OSS solo, and the collaboration**—on your curated set of problems.

Report per-problem results so we can see which outcomes change across conditions, and account for confounding factors as much as possible. Include transcripts where they provide evidence for your claims. This exercise is deliberately open-ended: we want to see how you turn a loosely specified question into a sound empirical investigation.

---

[1] The time cap is a safeguard against stalled runs.